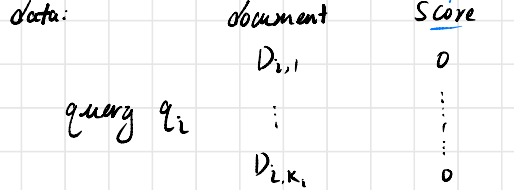
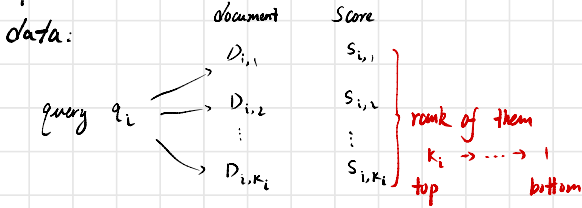


Learning to rank is to learn how to optimally order a list of items based on their relevance to a specific context. (e.g. a search query or a user profile)

There are mainly three types of losses $\begin{cases} \rightarrow \text{pointwise loss} \\ \rightarrow \text{pairwise loss} \\ \rightarrow \text{listwise loss} \end{cases}$

Here we cover basic of them and consider how to evaluate the results.

1. pointwise loss



$$L_{\text{pointwise}} = \sum_{i=1}^n \sum_{j=1}^{k_i} (f_\theta(q_i, D_{i,j}) - S_{i,j})^2$$

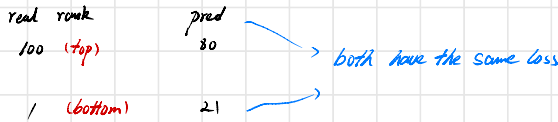
$$L_{\text{pointwise}} = - \sum_{i=1}^n \sum_{j=1}^{k_i} I(S_{i,j}=1) \log(\text{Sig} f_\theta(q_i, D_{i,j}))$$

pros: 1. Very easy to train

2. absolute scores can be useful, e.g. the probability of a user click.
conversion probability

Cons:

1. position agnostic.



However, getting the top ranking correct

2. Query imbalance problem

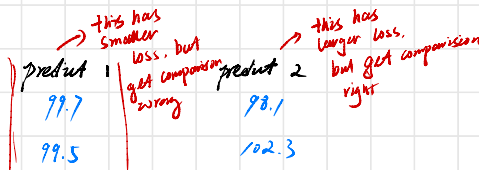
$\rightarrow$ Could try to mitigate the problem e.g. scale by number of documents

if a query is general, it could have more

Candidate documents $\rightarrow k_i$ is large so it dominates the loss. On the hand, the order for a general query is less important.

3. Ignoring Comparison

Say $S_{i,j_1} = 99$
 $S_{i,j_2} = 100$



the target we care about

3. Listwise loss

data	query	document	Score
	q_i	$D_{i,1}$	$S_{i,1}$
		$\vdots$	$\vdots$
		D_{i,K_i}	S_{i,K_i}

Compute the probability of each document to be ranked at the top

predicted probability

$$\hat{p}_{\theta}(D_{i,j} \text{ ranked at the top}) = \frac{\exp(f_{\theta}(q_i, D_{i,j}))}{\sum_{j=1}^{K_i} \exp(f_{\theta}(q_i, D_{i,j}))}$$

$\rightarrow D_{i,j} = K_i$

The real probability of each document ranked at the top

$$p(D_{i,j} \text{ ranked at the top}) = \frac{\exp(S_{i,j})}{\sum_{j=1}^{K_i} \exp(S_{i,j})}$$

Then the loss is Cross-entropy between the true and ideal distributions

$$\mathcal{L}_{\text{listwise}} = - \sum_{i=1}^N \sum_{j=1}^{K_i} p(D_{i,j}) \log(\hat{p}_{\theta}(D_{i,j}))$$

Training:

$$\text{batch} \left(\begin{pmatrix} (Q_i, D_{i,1}) \\ \vdots \\ (Q_i, D_{i,K_i}) \end{pmatrix} \right) \rightarrow \boxed{\begin{matrix} \text{model} \\ f_{\theta} \end{matrix}} \rightarrow \hat{z} = \begin{pmatrix} f_{\theta}(Q_i, D_{i,1}) \\ \vdots \\ f_{\theta}(Q_i, D_{i,K_i}) \end{pmatrix} \rightarrow \text{Compute cross entropy loss}$$

pros: it considers the rank of all documents in the query simultaneously and solves the position blindness problem for pointwise and pairwise approaches.

Cons: 1. it with focus heavily on top 1.

if there is 100 documents. top document take weight and have bad quality for top 2, top 3 etc.

$$\frac{\exp(100)}{\sum_{i=1}^{100} \exp(i)}$$